SESSION 1: GenAI Landscape

Segment 1: GenAI Stack

GENAI STACK

A GenAI system is far more than just a large language model. It is a layered architecture where each component plays a distinct role in transforming raw model capabilities into reliable, useful applications

The Five Layers of a GenAI Stack



Application Layer

User-facing experience and business logic

Chatbots, Copilots, Agents, Workflows

Tool Layer

Extends capabilities beyond text generation

API Calls, Code Execution, Web Browsing

Retrieval Layer

Brings external knowledge into context

RAG, Vector Database, Knowledge Graphs

Prompt Layer

Interface between human intent and model behaviour

System Prompts, Few-Shot Examples, Instructions

Model Layer

Raw intelligence – Pattern recognition, generation, reasoning

GPT-4, Claude, LLaMA, Gemini

THE STRATEGIC LANDSCAPE OF GENERATIVE AI DEPLOYMENT

Generative AI is transforming industries through two distinct paths: established use cases with manageable risks, and emerging applications promising massive potential but requiring more sophisticated frameworks

High-value lower-risk (Ready Now)

Verifiable Search & Summarisation
Speeding up information retrieval is low-risk when sources can be and verified

Accelerated Contract Drafting
Human-in-the-loop review ensures quality before publication while drastically increasing speed

30-50% Productivity Gains in Coding
Compilers and testing environments catch errors, making this a highly reliable use case

Emerging high-value (The next frontier)

Autonomous Agents & End-to-End Tasks
Significant potential for task completion is currently balanced by reliability and safety needs

Scientific & Medical Breakthroughs
High-stakes domains like discovery and documentation face challenges regarding hallucination and liability

Personalised Education at Scale
Emerging adaptive tutoring must address assessment validity and pedagogical quality

Operational Safety Nets

- **Customer Support:** Escalation paths exist for basic answer triage

Primary Challenges

- **Scientific Discovery:** Hallucination risk remains a barrier in high-stakes domains
- **Medical Documentation:** Clinician burden is reduced, but accuracy requirements are absolute

PROMPT LAYER: INTERFACE, CONTRACT, AND CONTROL SURFACE

The prompt layer is perhaps the most underappreciated component of modern GenAI systems. It serves three critical functions

- **Prompt as Interface:** The prompt is how humans communicate intent to machines. Unlike traditional APIs (predefined software interfaces with rigid input-output schemas), prompts accept natural language, making them incredibly flexible but also ambiguous. This flexibility is both a superpower and a liability

- **Prompt as Contract:** A well-designed system prompt establishes behavioural expectations
  - What the model should do (and refuse to do)
  - The persona and tone to maintain
  - Output format requirements
  - Boundaries and guardrails
  - Think of it as an informal SLA between the application developer and the model

- **Prompt as Control Surface:** Prompts are the primary mechanism for steering model behaviour without retraining. Through prompt engineering, developers can
  - Constrain outputs to specific formats (JSON, markdown, etc.)
  - Inject domain knowledge and context
  - Establish reasoning patterns (chain-of-thought, step-by-step)
  - Set safety boundaries

MODEL CAPABILITY VS SYSTEM RESPONSIBILITY

A critical distinction that many practitioners miss

Aspect	Model's Job	System's Job
Knowledge	General world knowledge	Domain-specific, current, proprietary data
Reasoning	Pattern-based inference	Verified, auditable logic
Accuracy	Probabilistically correct	Guaranteed correctness for critical paths
Safety	Following training-time alignment	Runtime guardrails, content filtering
Memory	Within-context only	Persistent state, user history
Actions	Suggesting actions	Executing actions safely

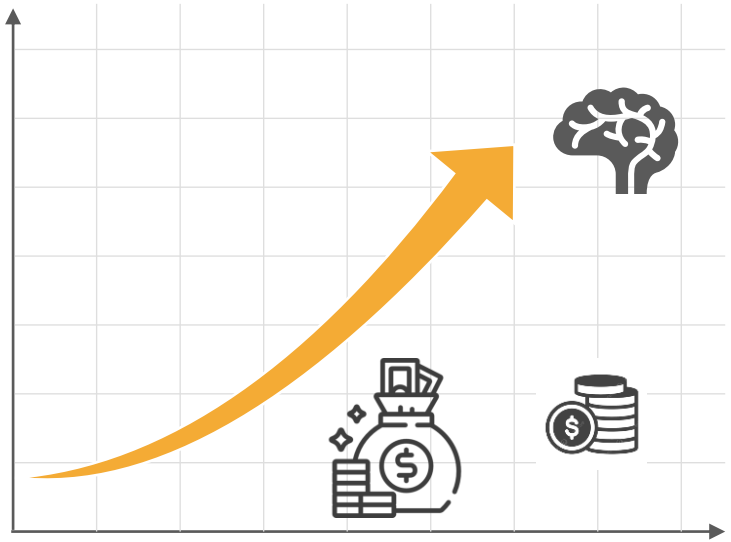
Models are probabilistic; *systems must be deterministic where it matters. Never rely on a model alone for authentication, authorisation, financial calculations, or safety-critical decisions*

Segment 2: Key Breakthroughs

SCALING: WHY SIZE CREATED EMERGENT CAPABILITIES

The scaling laws discovered by OpenAI and others showed that model capabilities improve predictably with

- More parameters
- More training data
- More compute



System Impact

- Larger models exhibit “emergent abilities” not present in smaller versions (in-context learning, chain-of-thought reasoning)
- This created a paradigm where a single general model could replace dozens of specialised ML models
- Systems could now rely on general-purpose reasoning rather than task-specific fine-tuning
- Infrastructure requirements shifted from “many small models” to “one large model serving many use cases”

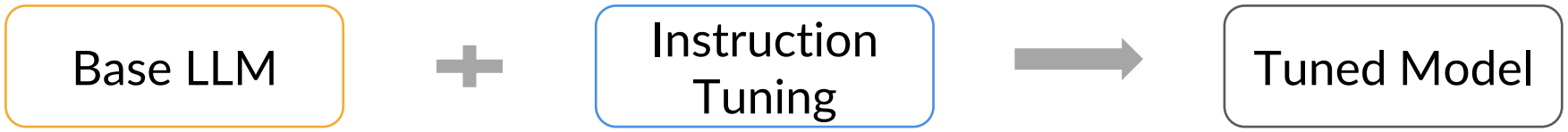
INSTRUCTION TUNING:
FROM COMPLETION TO CONVERSATION

Raw language models are trained to predict the next token; they complete text, not follow instructions. Instruction tuning (via RLHF, DPO, etc.) transformed models from

```
Input: "Write a poem about AI"
Raw Model Output: "is a popular request among..."
Instruction-Tuned: "Here's a poem about AI: Silicon dreams awaken..."
```

System Impact

- Models became directly usable without complex prompt engineering
- Reduced the “prompt engineering tax” for simple applications
- Made chat interfaces viable as a primary interaction paradigm
- Created the expectation that models should be “helpful, harmless, and honest”

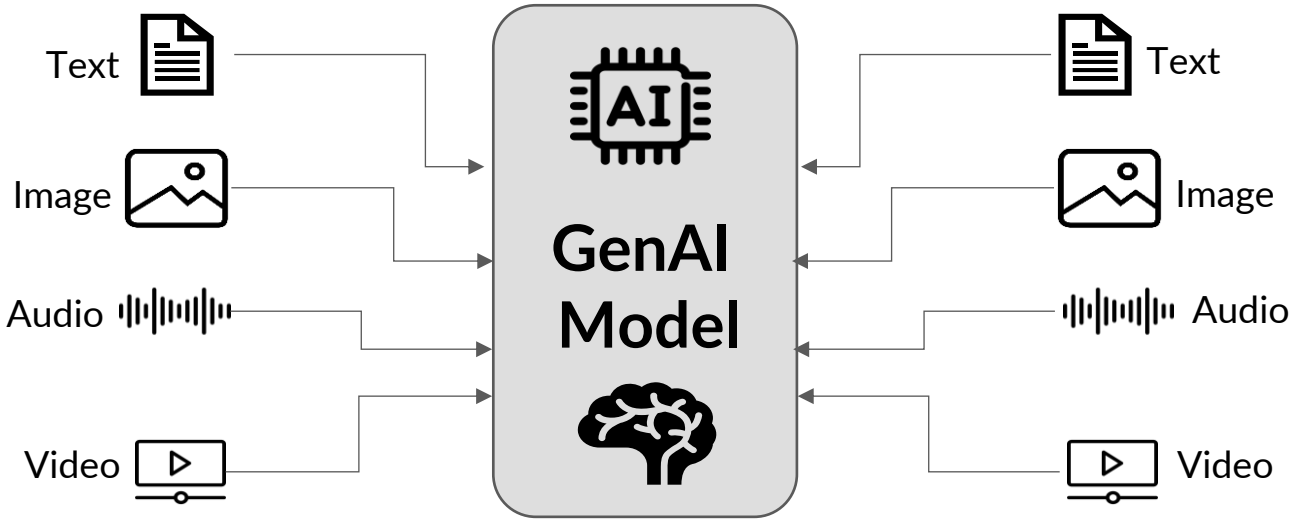


MULTIMODALITY: BREAKING THE TEXT BARRIER

Modern GenAI systems process images, audio, video, and code alongside text

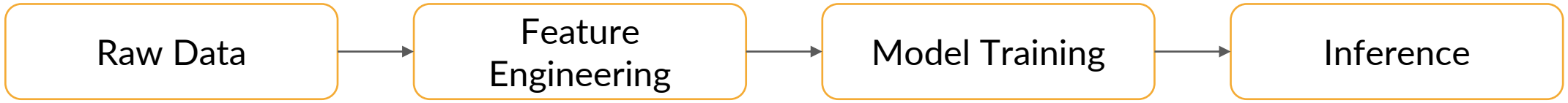
System Impact

- Single models can now replace entire pipelines (OCR → extraction → summarisation)
- Applications can accept any input modality that users naturally produce
- Unified architectures simplify system design
- Creates new failure modes (e.g., hallucinating image content, misinterpreting visual context)



WHY PROMPTS REPLACED FEATURE ENGINEERING

Traditional ML Pipeline



GenAI Pipeline



The shift is profound

- **Before:** Data scientists spent 80% of their time on feature engineering
→ **Now:** Prompt design encodes domain knowledge directly

- **Before:** Each task required a custom model
→ **Now:** One model, many prompts

This doesn't eliminate the need for expertise; it shifts it. Feature engineering expertise becomes prompt engineering, context curation, and system design expertise

Segment 3: Impacts and Use Cases

REAL WORLD GENAI USE CASES

High-Value, lower-Risk Applications

Use Case		Value Delivered	Why it Works
	Code Assistance	30-50% productivity gains	Errors are caught by compilers/tests
	Content Drafting	Faster first drafts	Human review before publication
	Search & Summarisation	Faster information retrieval	Sources can be cited and verified
	Translation	Near-instant multilingual content	Quality is verifiable
	Customer Support Triage	Faster routing, basic answers	Escalation paths exists

Emerging High-Value Applications

Use Case		Potential	Challenge
	Autonomous Agents	End-to-end task completion	Reliability, Safety, Recovery
	Scientific Discovery	Hypothesis generation, literature review	Hallucination risk in high-stakes domains
	Personalised Education	Adaptive tutoring at scale	Assessment validity, Pedagogy quality
	Medical Documentation	Reducing clinician burden	Accuracy requirements, Liability

COMMON FAILURE PATTERNS

01 The Hallucination Problem

Models confidently generate false information. This isn't a bug; it's intrinsic to how language models work

- **Pattern:** System presents model output as fact without verification
- **Fix:** Ground responses in retrieved documents, add confidence indicators, and enable source checking

02 Context Window Abuse

Stuffing too much information into context, expecting perfect recall

- **Pattern:** "Just put all the documents in the context"
- **Fix:** Intelligent retrieval, chunking strategies, hierarchical summarisation

03 The Prompt Injection Attack

Malicious inputs that hijack model behaviour

- **Pattern:** User input concatenated directly into prompts
- **Fix:** Input sanitisation, output validation, privilege separation

04 Capability Overestimation

Assuming the model can do things it fundamentally cannot

- **Pattern:** Using LLMs for precise calculation, real-time data, or guaranteed correctness
- **Fix:** Hybrid architectures with specialised tools for deterministic tasks

MISUSE, OVERRELIANCE, AND DEPLOYMENT RISKS



Misuse Risks

- Generating misinformation at scale
- Creating deceptive content (phishing, scams)
- Automating harassment or manipulation
- Cheating and plagiarism
- Bypassing security controls



Overreliance Risks

- Skill atrophy in human operators
- Automation bias (trusting AI over human judgment)
- Loss of institutional knowledge
- Single points of failure



Deployment Risks

- Bias amplification (models reflect training data biases)
- Privacy leakage (models may memorise training data)
- Unpredictable behaviour at scale
- Regulatory and compliance violations
- Reputational damage from public failures

MITIGATION FRAMEWORK

1. Human-in-the-loop for high-stakes decisions



Human-in-the-loop
for high-stakes decisions



2. Layered Defences (Input/Output Filtering, Monitoring)



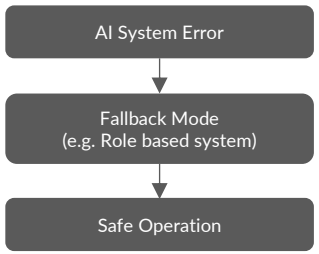
Layered Defences
(Input/Output Filtering,
Monitoring)



3. Graceful degradation when AI fails



Graceful degradation
when AI fails



4. Audit trails for accountability



Audit trails for
accountability



5. Regular Evaluation Against Benchmark & Real-World Performance



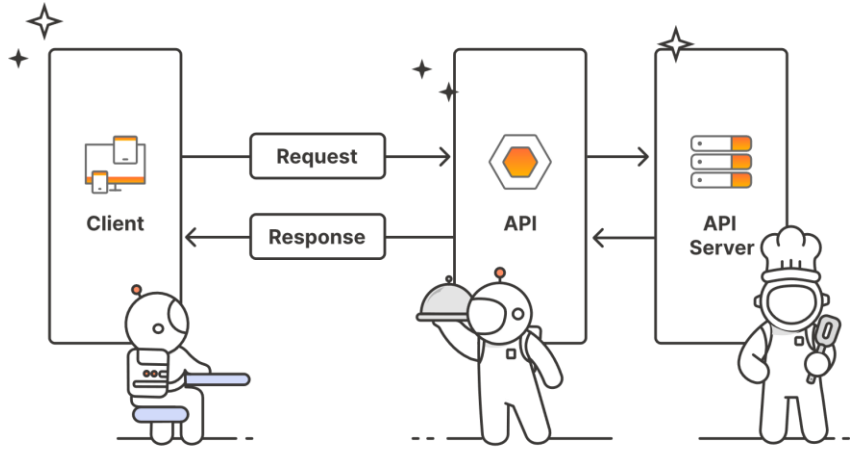
Regular Evaluation Against
Benchmark & Real-World
Performance



Segment 4: Protocols and Interoperability

WHAT IS AN APPLICATION PROGRAMMING INTERFACE (API)?

- An API (Application Programming Interface) defines how one software system can request functions or data from another system
- Exposes predefined operations (endpoints) with structured inputs and outputs
- Follows a request–response interaction model
- Assumes deterministic behaviour and predictable system responses
- Widely used to integrate services such as authentication, payments, data access, and notifications



TYPES OF APIs (BASED ON ACCESS LEVEL)

APIs can be categorised based on who is allowed to use them and how they are exposed

Private APIs

- Used to connect software components and services within a single organisation
- Not exposed to third-party developers
- Common in microservice-based and internal platform architectures
- **Use cases**
  - Authentication
  - Data pipelines
  - Service orchestration

Public APIs

- Exposed to external developers to access an organisation's services or data
- Governed by usage policies, rate limits, and pricing models
- Enable platform ecosystems and third-party integrations
- **Use cases**
  - Payments
  - Mapping
  - Messaging

Partner APIs

- Shared selectively with authorised partner organisations
- Enable controlled cross-organisation data and service access
- Secured using authentication and contractual access controls
- **Use cases**
  - Banking integration
  - Logistics
  - Data sharing

COMMON API ARCHITECTURAL STYLES

APIs also differ in how interactions are structured at an architectural level

- **REST:** Resource-oriented APIs using standard HTTP methods for stateless request–response interactions
- **SOAP:** XML-based messaging protocol used in structured and security-heavy enterprise environments
- **GraphQL:** Query-based API style that allows clients to request exactly the data they need from a single endpoint
- **Webhooks (Event-driven APIs):** Systems notify other systems automatically when specific events occur
- **gRPC (RPC-style APIs):** Enables remote procedure calls between distributed services using strongly typed contracts

WHY GENAI SYSTEMS NEED PROTOCOLS BEYOND APIs

Traditional APIs work well when

- Inputs and outputs are well-defined
- Operations are stateless or have simple state
- The client knows exactly what it wants
- Traditional systems are more deterministic

GenAI systems break these assumptions

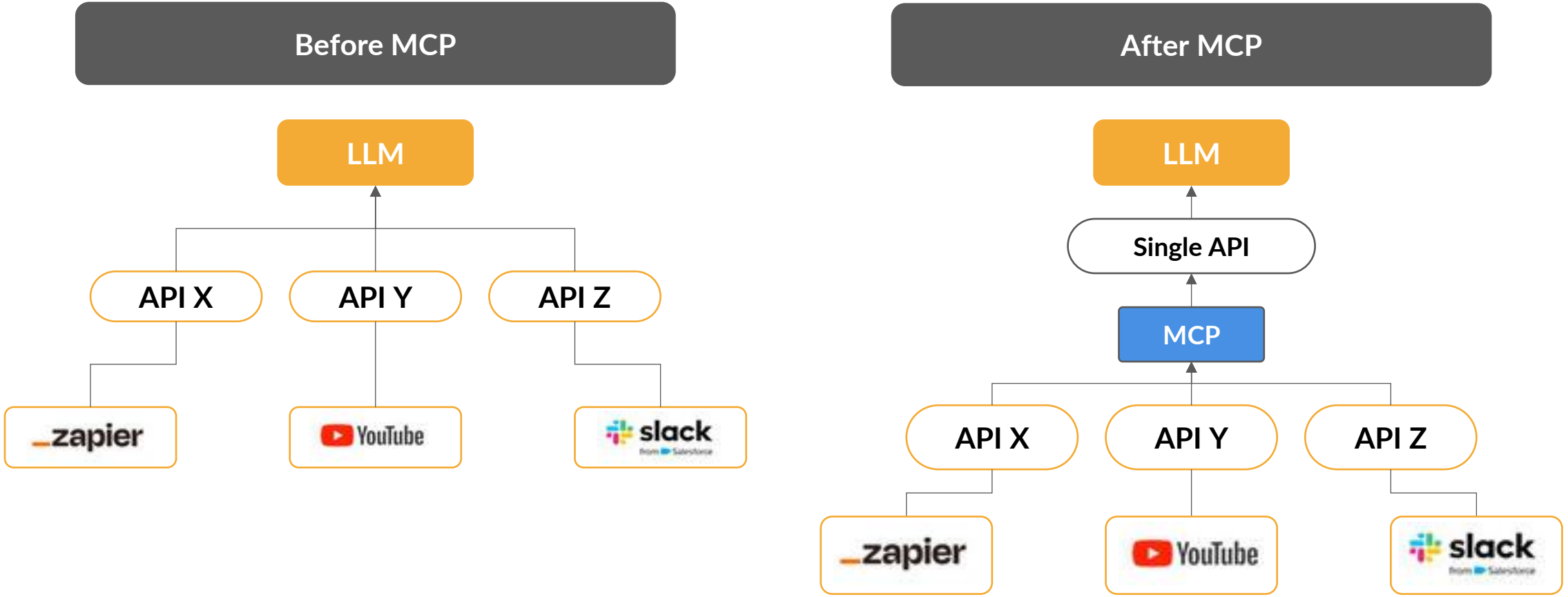
- Context is dynamic and application-specific
- Models need access to tools, memory, and external data
- Conversations have state that spans multiple interactions
- Different components (models, tools, memory) need to interoperate

The Integration Problem

- Without protocols, every GenAI application becomes a custom integration project that requires custom tool definitions for each model
- Custom context formatting
- Custom memory management
- No portability between models or frameworks

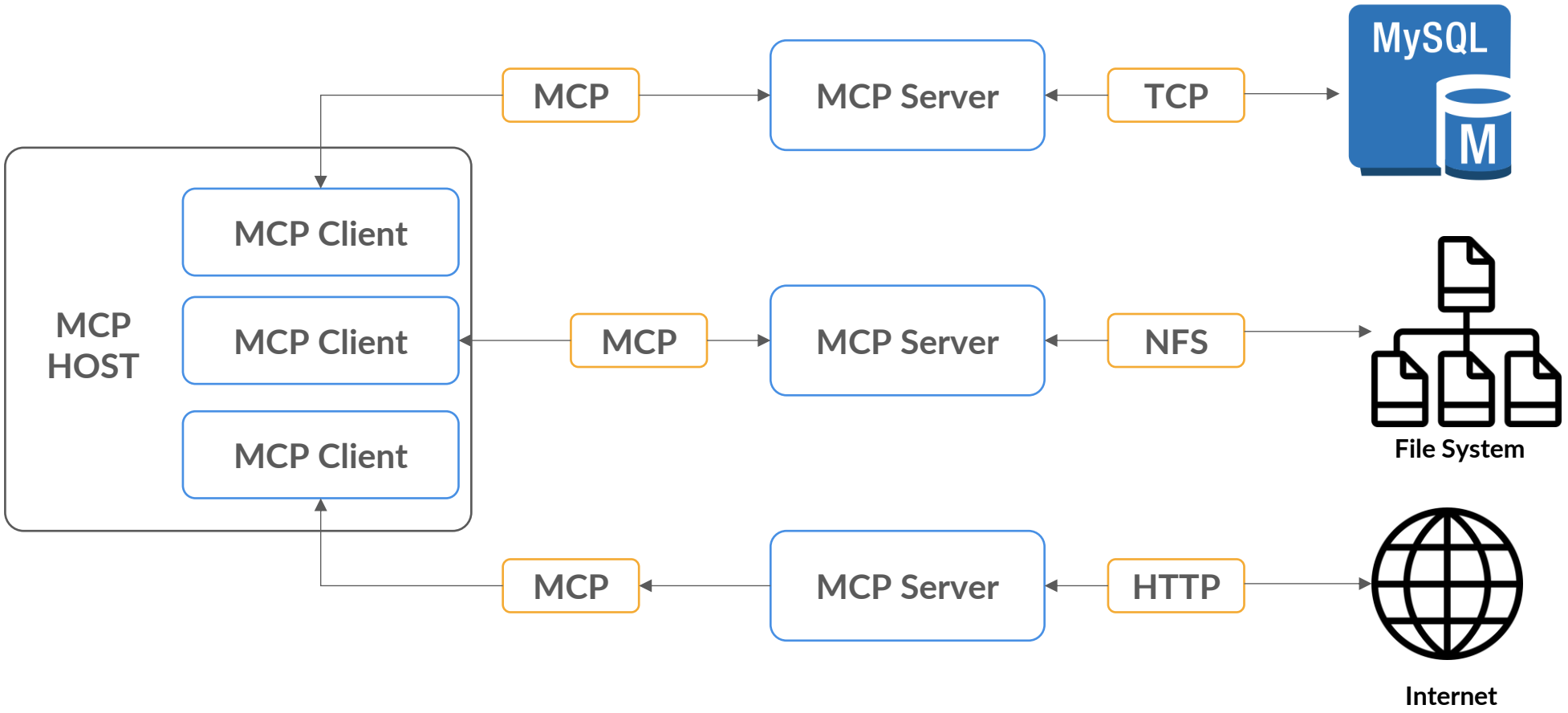
MODEL CONTEXT PROTOCOL (MCP)

MCP is an open protocol (developed by Anthropic) that standardises how GenAI applications connect to external data sources, tools, and services



MCP CORE CONCEPT

MCP separates concerns into **Hosts** (AI applications), **Clients** (protocol connectors), and **Servers** (capability providers)



MODEL INFERENCE VS CONTEXT & TOOL EXECUTION

This is a crucial architectural distinction

Aspect	Model Inference	Context & Tool Execution
What	Generating text, reasoning	Reading files, calling APIs, querying databases
Where	Model provider's infrastructure	User's environment or trusted servers
Trust	Model sees everything in context	Tools have controlled, scoped access
State	Stateless (context is input)	Stateful (persistent data, side effects)
Speed	Seconds (inference time)	Milliseconds to minutes (I/O bound)

Why This Matters

Models shouldn't have direct access to your systems. MCP creates a controlled boundary where

- Models request actions through a defined interface
- The execution environment validates and performs actions
- Results are returned to the model as context

PROBLEMS MCP IS DESIGNED TO SOLVE

1

Context Provisioning

- Problem: Every app builds custom integrations to fetch relevant context
- MCP Solution: Standardised “Resources” that any host can access

2

Tool Execution

- Problem: Tool definitions differ across models and frameworks
- MCP Solution: Unified tool schema with consistent invocation patterns

3

Memory & State

- Problem: No standard for persistent memory across sessions
- MCP Solution: State management capabilities in the protocol

4

Model vs. System Boundaries

- Problem: Unclear what the model controls vs. what the system controls
- MCP Solution: Explicit separation between inference and execution

5

Interoperability

- Problem: Tools built for one model/framework don't work with others
- MCP Solution: Model-agnostic protocol that any host can implement

HOW MCP SUPPORTS REUSABLE AND COMPOSABLE SYSTEMS

01

Composability Benefits: Build Once, Use Everywhere

- MCP servers work with Claude Desktop
- MCP servers work with VS Code + Continue
- MCP servers work with Custom Agent Frameworks
- MCP servers work with Future Applications

02

Reusability Patterns

- **Server Reuse:** Write one MCP server for a capability (e.g., database access), use it across all your AI applications
- **Host Flexibility:** Switch between AI providers without rebuilding integrations
- **Community Ecosystem:** Share and consume MCP servers from the community

03

Agent Portability: Agents built on MCP can theoretically

- Move between different model providers
- Access the same tools regardless of host application
- Maintain consistent behaviour across environments

MCP BENEFITS AND LIMITATIONS

Benefit	Description
Standardisation	Common protocol reduces custom integration work
Security Boundaries	Clear separation between model and execution environment
Ecosystem	Growing library of pre-built servers
Flexibility	Transport-agnostic (stdio, HTTP+SSE)
Local-First	Servers can run locally, keeping data on-premises

Limitation	Description
Maturity	Protocol is still evolving; breaking changes possible
Adoption	Not yet universal; many platforms don't support MCP
Complexity	Adds architectural overhead for simple use cases
Performance	Protocol overhead for high-frequency tool calls
Discovery	No built-in mechanism for finding/registering server

COMMON MCP MISUSE AND RISKS

Over-Engineering Simple Tasks

- Using MCP for a single integration that won't be reused
- Adding protocol complexity when a simple API call suffices

Security Anti-Patterns

- Exposing MCP servers to untrusted networks
- Granting excessive permissions to servers
- Not validating tool inputs/outputs

Performance Pitfalls

- Synchronous tool calls blocking model inference
- Not implementing proper timeouts
- Fetching too much data through resources

Governance Gaps

- No audit logging for tool invocations
- Unclear ownership of MCP servers
- No versioning strategy for server capabilities

WHEN MCP IS APPROPRIATE VS. UNNECESSARY

Use MCP When

- Building applications that will integrate multiple AI models
- Creating reusable tool/context servers for your organisation
- You need clear security boundaries between the model and execution
- Building agents that need to access multiple data sources
- You want to participate in the MCP ecosystem

Skip MCP When

- Simple, single model application with one or two integrations
- Prototyping or proof-of-concept (add later if needed)
- Latency-critical applications where protocol overhead matters
- The host application doesn’t support MCP
- You're building closed, proprietary systems with no reuse intent